Mushroom Toxicity Classification Project

Tommy Yu

2025-03-17

Introduction

When we think of a poisonous mushroom, what comes to mind first? Many assume that bright colors indicate toxicity, while others believe shape or form plays a key role. However, determining a mushroom's toxicity requires analyzing multiple factors; making a mistake can be fatal. Mushrooms can vary in color, shape, and other attributes, but which traits are truly the best indicators of toxicity? What common misconceptions about these attributes exist?

In this project, we aim to answer these questions by training a classification algorithm using the UCI Mushroom Dataset, which categorizes mushrooms as poisonous or edible based on various physical attributes. Our primary goal is to minimize False Positives, as mistaking a poisonous mushroom for an edible one is far more dangerous than mistaking an edible mushroom for a poisonous one. However, we still want to be able to properly identify edible mushrooms, so we should still consider the True Positive Rate when evaluating the model. A low False Positive Rate ensures safety and the True Positive Rate becomes a secondary objective.

Loading Data

```
suppressMessages(library(readr))
suppressMessages(library(tidyverse))
suppressMessages(library(ISLR))
suppressMessages(library(glmnet))
suppressMessages(library(tree))
suppressMessages(library(maptree))
suppressMessages(library(randomForest))
suppressMessages(library(gbm))
suppressMessages(library(ROCR))
mushrooms <- read_csv("C:/Users/tommy/Desktop/homework/131/mushrooms.csv",show_col_types= FALSE)
colnames(mushrooms) <- c('class','cap_shape','cap_surface','cap_color','bruises','odor','gill_attachmen

# Turning categorical variables into factors
mushrooms <- mushrooms %>% mutate(poisonous = as.factor(if_else(class=='p','Poisonous','Edible')))
mushrooms <- mushrooms %>% mutate_all(as.factor)

# Cleaning dataset, removing variables with missing values and removing odor
mushrooms <- subset(mushrooms, select=-c(veil_type,stalk_root,class,odor))
mushrooms <- na.omit(mushrooms)

# Splitting the dataset to about 80% train 20% test
set.seed(123)
```

```r
train_ind <- sample(seq_len(nrow(mushrooms)),size=6500)
train <- mushrooms[train_ind,]
test <- mushrooms[-train_ind,]
```

About the Dataset

The mushroom dataset is featured in UCI's Machine Learning repository. It features 22 categorical variables and the dependent variable is class, which determines whether the mushroom is poisonous or edible. The dataset has some missing values in the variable veil_type and stalk color, which we have removed from our models.

Model Selection

We decided to use decision trees and its variants to model our scenario. It made the most sense out of all the classification models, check the attribute of the unidentified mushroom and make a decision out of that attribute. We made multiple models using decision trees, boosting and random forests to see just how much information you really need to confidently classify a mushroom. We featured a model with all the variables, a model with just the variables related to the color of the mushroom, and a model with just the variables related to the shape of the mushroom.

Disclaimer

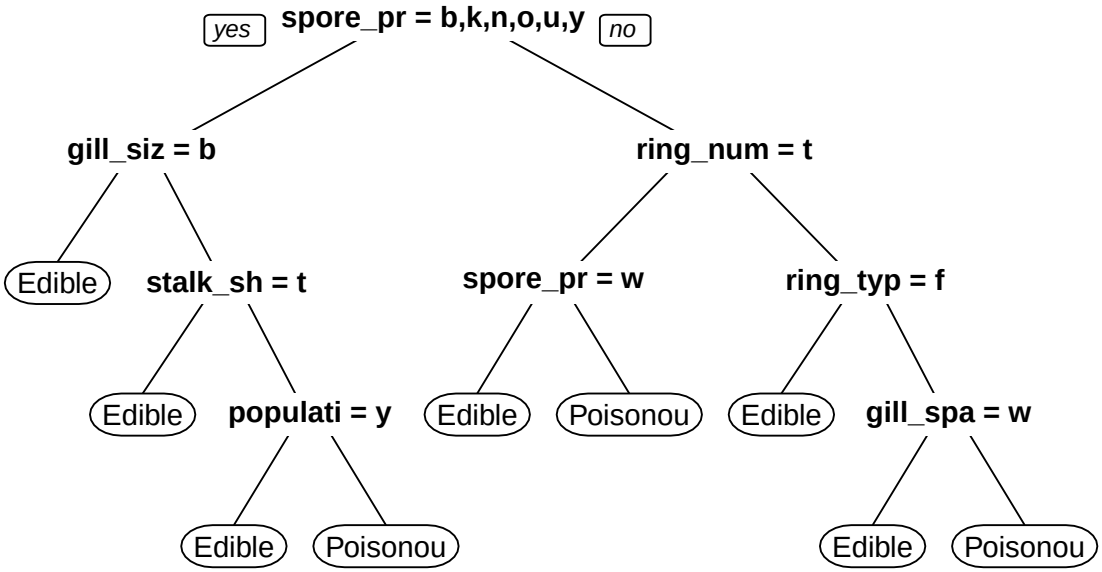
The variable Odor, has a 99% accuracy prediction when it comes to poisonous/edible. Not only would this make for an uninteresting project, it would also not make too much sense in real life applications. Mushrooms can emit toxic fumes so smelling them might not be the best idea. As a result, the Odor variable has been omitted from all models. Additionally, note that spore print color is excluded from the color models. This is because spore prints are not related to the appearance of the mushroom and you have to press the mushroom onto a piece of paper to obtain the spore print color.

The Models

Basic Decision Tree

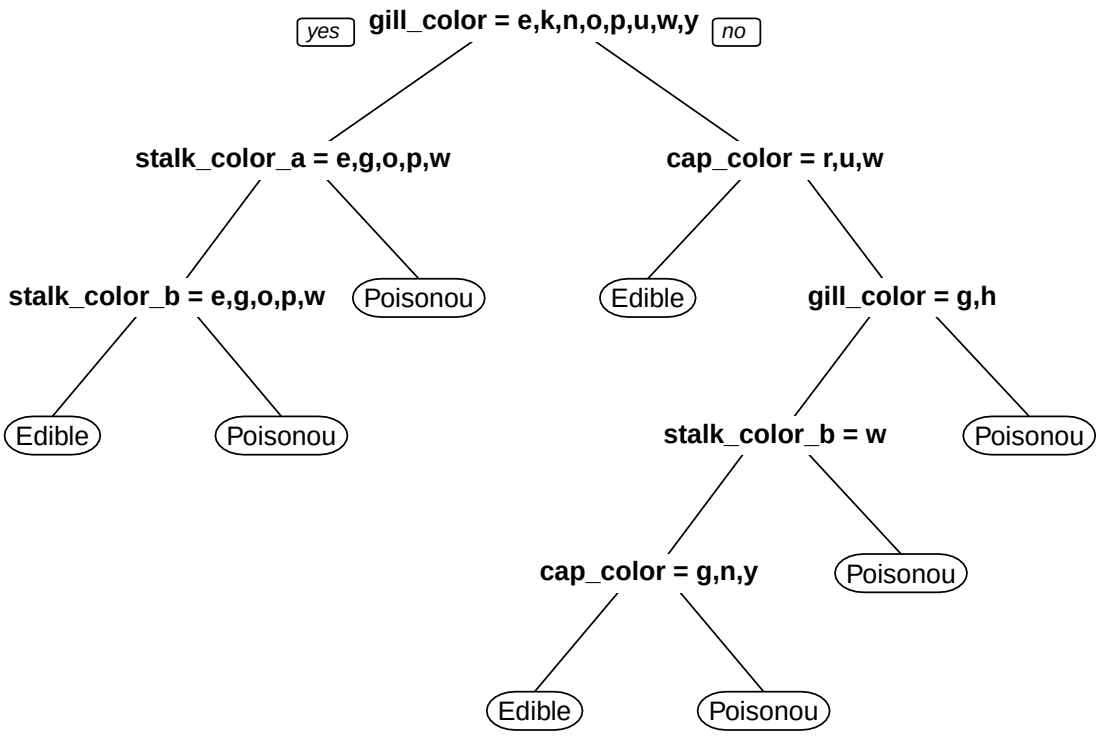
```r
library(rpart.plot)
suppressMessages(library(vip))

set.seed(123)
tree.mushroom <- rpart(poisonous ~., data=train)
prp(tree.mushroom)
```



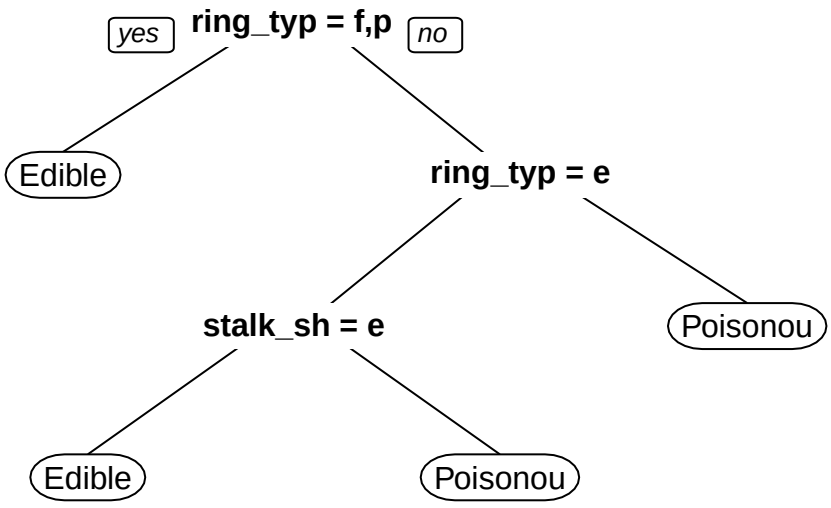

```
mushroom.trees <- predict(tree.mushroom, type='class',newdata=test)
mushroomCM <- table(test$poisonous, mushroom.trees)

set.seed(123)
tree.mushroomColor <- rpart(poisonous ~ cap_color+gill_color+stalk_color_below+stalk_color_above+veil_c
prp(tree.mushroomColor)
```




```
color.trees <- predict(tree.mushroomColor, type='class',newdata=test)
colorCM <- table(test$poisonous, color.trees)

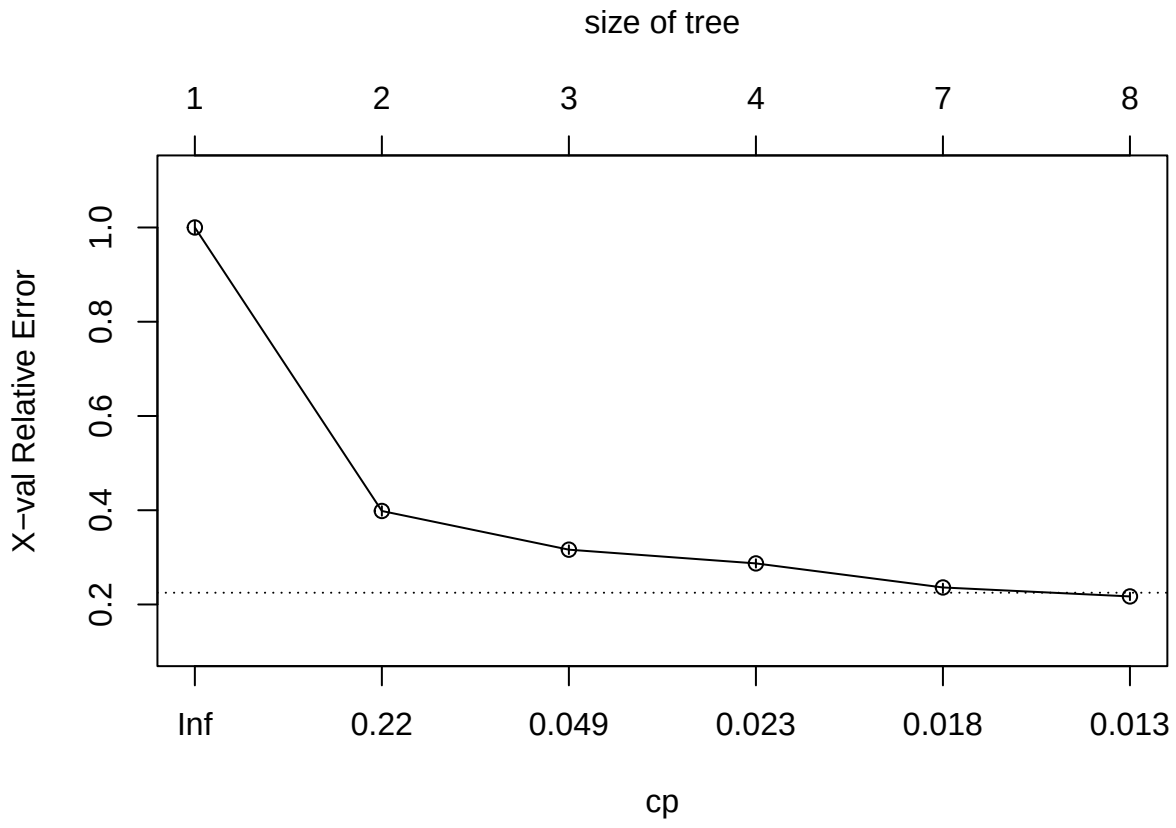
set.seed(123)
tree.mushroomShape <- rpart(poisonous ~ cap_shape + stalk_shape + ring_type,data=train)
prp(tree.mushroomShape)
```



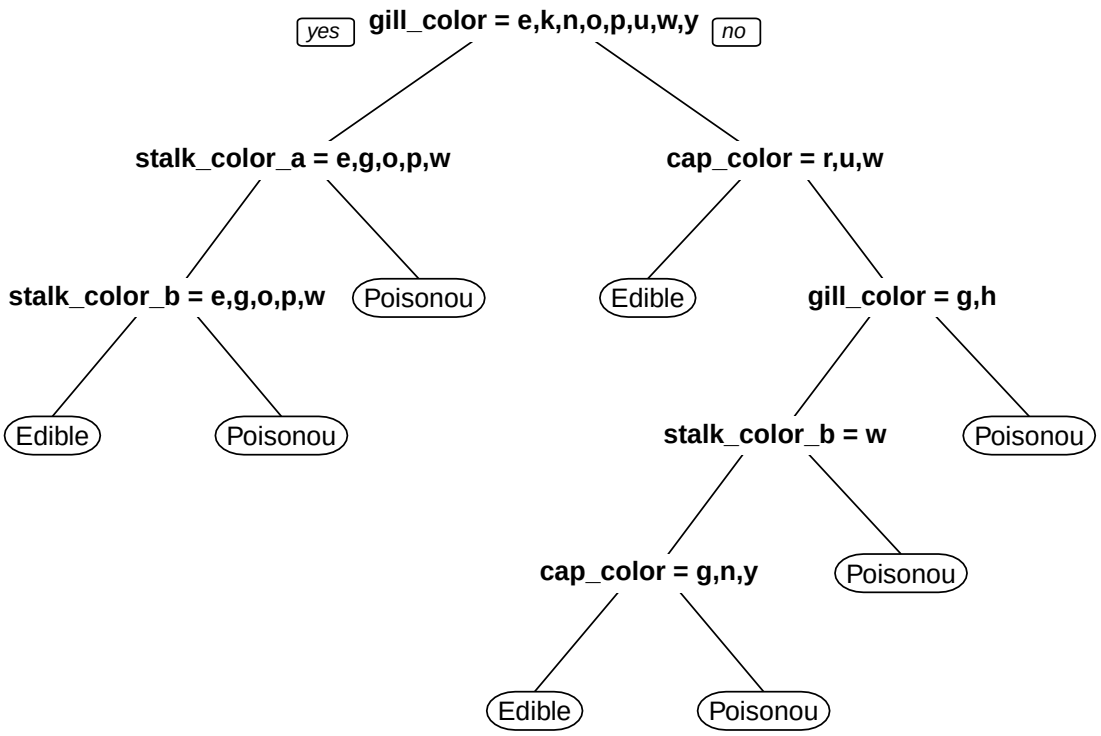

```
shape.trees <- predict(tree.mushroomShape, type='class', newdata=test)
shapeCM <- table(test$poisonous, shape.trees)
```

Pruning

```
plotcp(tree.mushroomColor)
```



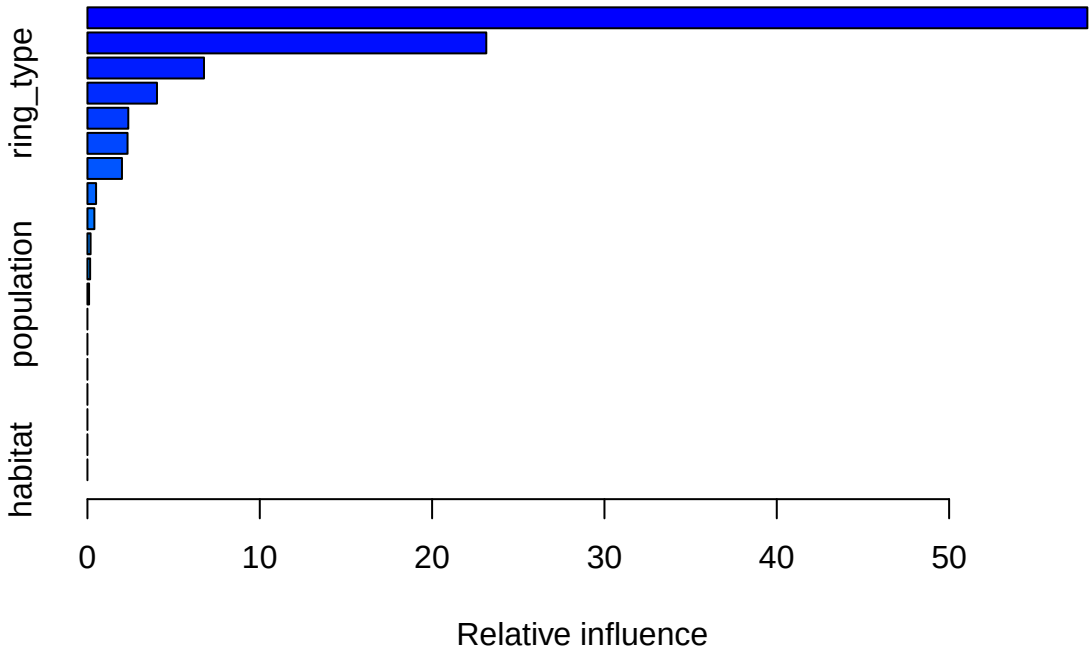

```
optimalCP <- tree.mushroomColor$cptable[which.min(tree.mushroomColor$cptable[,'xerror']),'CP']
pruned_tree <- prune(tree.mushroomColor,optimalCP)
prp(pruned_tree)
```



As we can see, the pruned tree is exactly the same as the unpruned tree. This indicates that we do not have an issue with overfitting.

Boosting

```
set.seed(123)
boost.mushroom <-gbm(ifelse(poisonous=='Poisonous',1,0)~.,data=train,n.trees=1000,shrinkage=0.01,distri
summary(boost.mushroom)
```



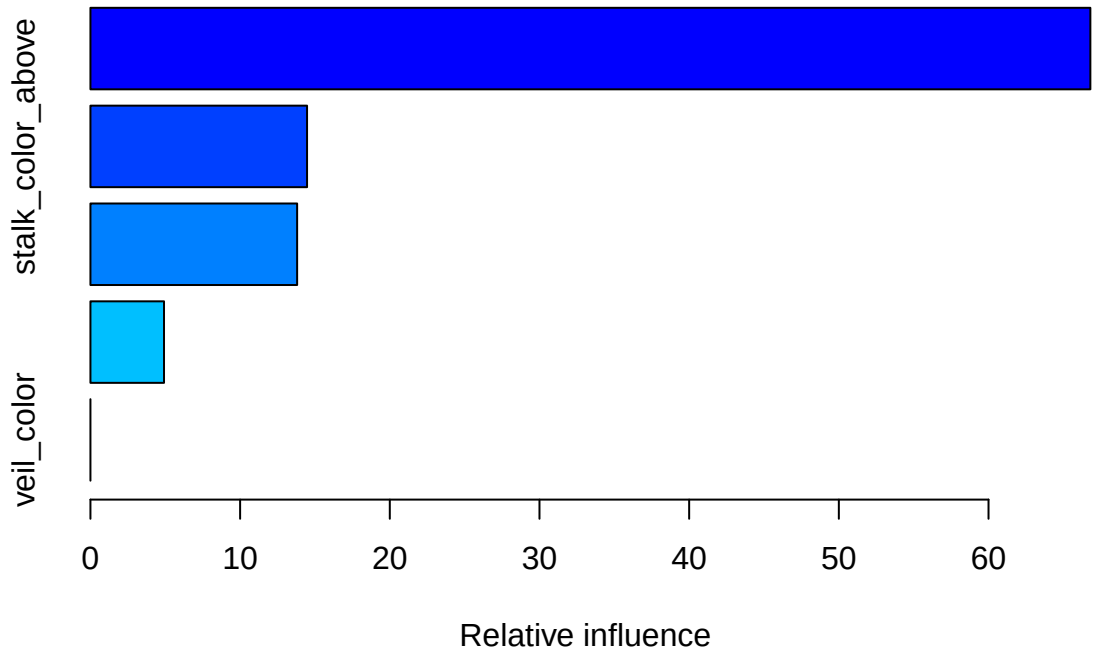

```
##                                   var     rel.inf
## spore_print_color     spore_print_color 58.02145858
## gill_size                   gill_size 23.14333038
## gill_color                 gill_color  6.76371116
## ring_type                   ring_type  4.03511288
## gill_spacing             gill_spacing  2.37030267
## stalk_surface_above stalk_surface_above  2.32742726
## ring_number               ring_number  2.00362041
## cap_color                   cap_color  0.50073704
## cap_shape                   cap_shape  0.40187225
## stalk_color_below     stalk_color_below  0.18186955
## stalk_color_above     stalk_color_above  0.15620737
## population                 population  0.09435045
## cap_surface               cap_surface  0.00000000
## bruises                       bruises  0.00000000
## gill_attachment         gill_attachment  0.00000000
## stalk_shape                 stalk_shape  0.00000000
## stalk_surface_below stalk_surface_below  0.00000000
## veil_color                 veil_color  0.00000000
## habitat                       habitat  0.00000000
```

```
mushroom.boost <- predict(boost.mushroom,type='response',newdata=test)
```

```
## Using 1000 trees...
```

```
mushroom.boost <- ifelse(mushroom.boost >= 0.5, 'Poisonous','Edible')
boostCM <- table(test$poisonous,mushroom.boost)


set.seed(123)
boost.mushroomColor <- gbm(ifelse(poisonous=='Poisonous',1,0) ~cap_color+gill_color+stalk_color_below+s
summary(boost.mushroomColor)
```



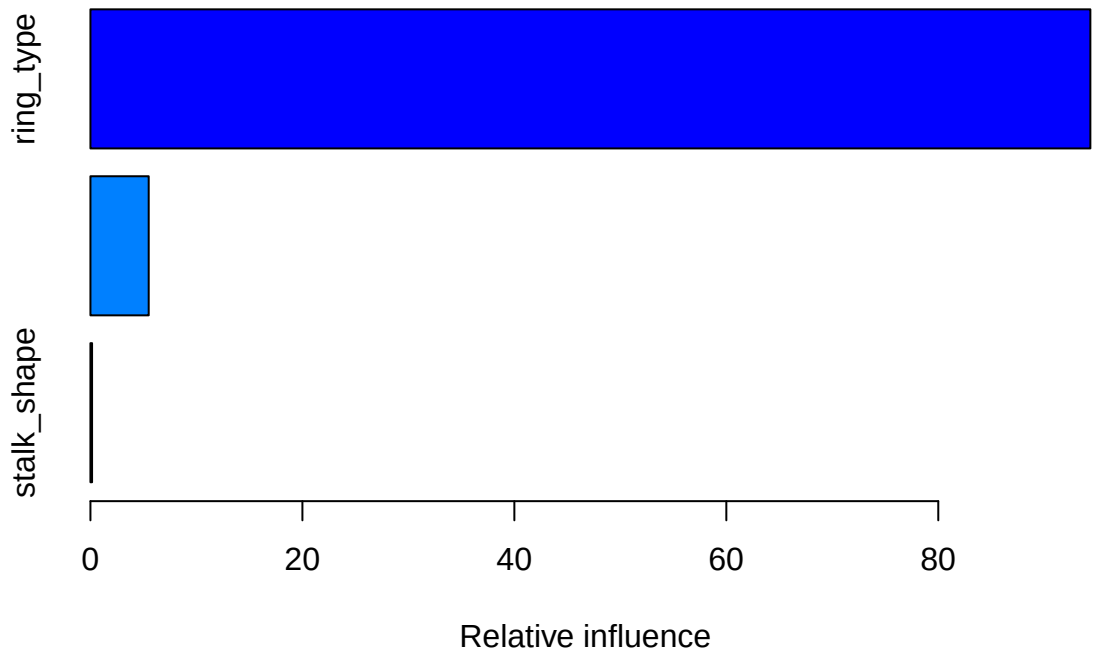

```
##                                 var   rel.inf
## gill_color               gill_color 66.802947
## stalk_color_above stalk_color_above 14.471430
## stalk_color_below stalk_color_below 13.809490
## cap_color                 cap_color  4.916133
## veil_color               veil_color  0.000000
```

```
mushroomColor.boost <- predict(boost.mushroomColor, type='response',newdata=test)
```

```
## Using 1000 trees...
```

```
mushroomColor.boost <- ifelse(mushroomColor.boost >= 0.5, 'Poisonous','Edible')
boostColorCM <- table(test$poisonous,mushroomColor.boost)

set.seed(123)
boost.mushroomShape <- gbm(ifelse(poisonous=='Poisonous',1,0)~cap_shape + stalk_shape + ring_type,data=
summary(boost.mushroomShape)
```




```
##                     var    rel.inf
## ring_type     ring_type 94.3494456
## cap_shape     cap_shape  5.4968435
## stalk_shape stalk_shape  0.1537109
```

```
mushroomShape.boost <- predict(boost.mushroomShape, type='response',newdata=test)
```

```
## Using 1000 trees...
```

```
mushroomShape.boost <- ifelse(mushroomShape.boost >= 0.5, 'Poisonous','Edible')
boostShapeCM <- table(test$poisonous,mushroomShape.boost)
```

Random Forest

```
# Assign column names (as per the dataset documentation)
library(caret)
```

```
## Loading required package: lattice
```

```
##
## Attaching package: 'caret'
```

```
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
# Set seed for reproducibility
set.seed(123)

# Train a Random Forest model
rf_model <- randomForest(poisonous ~ ., data = train, ntree = 500, mtry = 3, importance = TRUE)

# Print model summary
print(rf_model)
```

```
##
## Call:
##  randomForest(formula = poisonous ~ ., data = train, ntree = 500,      mtry = 3, importance = TRUE)
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 3
##
##         OOB estimate of  error rate: 0%
## Confusion matrix:
##           Edible Poisonous class.error
## Edible      3345         0           0
## Poisonous      0      3155           0
```

```r
# Make predictions on the test dataset
rf_predictions <- predict(rf_model, test)

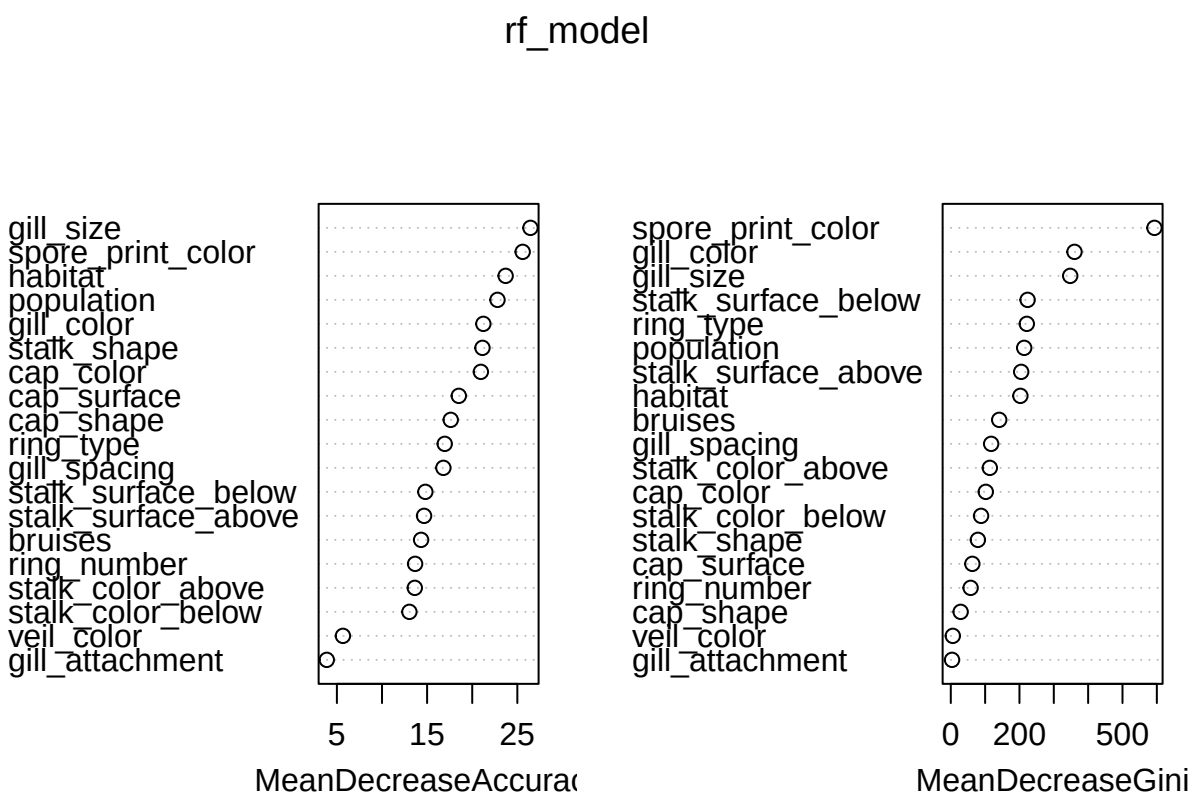
# Evaluate the model using a confusion matrix
conf_matrix <- confusionMatrix(rf_predictions, test$poisonous)

# Print confusion matrix and accuracy
print(conf_matrix)
```

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction  Edible Poisonous
##   Edible       863         0
##   Poisonous      0       761
##
##                Accuracy : 1
##                  95% CI : (0.9977, 1)
##     No Information Rate : 0.5314
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##             Sensitivity : 1.0000
##             Specificity : 1.0000
```

```
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 1.0000
##              Prevalence : 0.5314
##          Detection Rate : 0.5314
##    Detection Prevalence : 0.5314
##       Balanced Accuracy : 1.0000
##
##        'Positive' Class : Edible
##
```

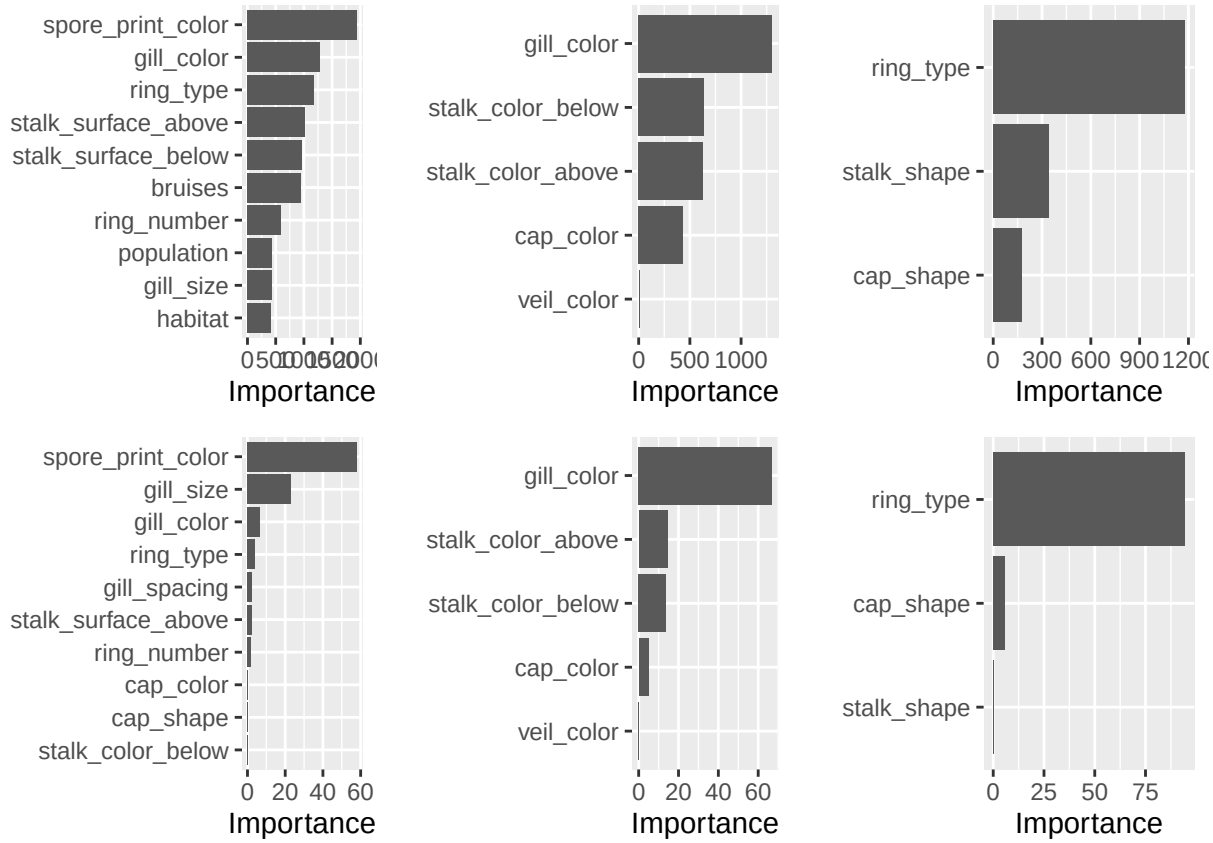
```r
# Plot variable importance
varImpPlot(rf_model)
```



Model Analysis

```r
suppressMessages(library(gridExtra))
mushroomImp <- vip::vip(tree.mushroom,num_features=10)
colorImp <- vip::vip(tree.mushroomColor,num_features=10)
shapeImp <- vip::vip(tree.mushroomShape,num_features=10)
boostMushroomImp <- vip::vip(boost.mushroom,num_features=10)
boostColorImp <- vip::vip(boost.mushroomColor,num_features=10)
boostShapeImp <- vip::vip(boost.mushroomShape,num_features=10)
```

```
grid.arrange(mushroomImp,colorImp,shapeImp,boostMushroomImp,boostColorImp,boostShapeImp,ncol=3)
```



ROC and AUC

```
suppressMessages(library(pROC))

pred_probs_tree_mushroom <- predict(tree.mushroom, test, type = "prob")[, 2]
roc_tree_mushroom <- roc(test$poisonous, pred_probs_tree_mushroom)
```

```
## Setting levels: control = Edible, case = Poisonous
```

```
## Setting direction: controls < cases
```

```
auc_tree_mushroom <- auc(roc_tree_mushroom)

pred_probs_tree_mushroomColor <- predict(tree.mushroomColor, test, type = "prob")[, 2]
roc_tree_mushroomColor <- roc(test$poisonous, pred_probs_tree_mushroomColor)
```

```
## Setting levels: control = Edible, case = Poisonous
## Setting direction: controls < cases
```

```r
auc_tree_mushroomColor<- auc(roc_tree_mushroomColor)

pred_probs_tree_mushroomShape <- predict(tree.mushroomShape, test, type = "prob")[, 2]
roc_tree_mushroomShape <- roc(test$poisonous, pred_probs_tree_mushroomShape)
```

```
## Setting levels: control = Edible, case = Poisonous
## Setting direction: controls < cases
```

```r
auc_tree_mushroomShape <- auc(roc_tree_mushroomShape)

pred_probs_boost_mushroom <- predict(boost.mushroom, test, type = "response")
```

```
## Using 1000 trees...
```

```r
roc_boost_mushroom <- roc(test$poisonous, pred_probs_boost_mushroom)
```

```
## Setting levels: control = Edible, case = Poisonous
## Setting direction: controls < cases
```

```r
auc_boost_mushroom <- auc(roc_boost_mushroom)

pred_probs_boost_mushroomColor <- predict(boost.mushroomColor, test, type = "response")
```

```
## Using 1000 trees...
```

```r
roc_boost_mushroomColor <- roc(test$poisonous, pred_probs_boost_mushroomColor)
```

```
## Setting levels: control = Edible, case = Poisonous
## Setting direction: controls < cases
```

```r
auc_boost_mushroomColor <- auc(roc_boost_mushroomColor)

pred_probs_boost_mushroomShape <- predict(boost.mushroomShape, test, type = "response")
```

```
## Using 1000 trees...
```

```r
roc_boost_mushroomShape <- roc(test$poisonous, pred_probs_boost_mushroomShape)
```

```
## Setting levels: control = Edible, case = Poisonous
## Setting direction: controls < cases
```

```r
auc_boost_mushroomShape <- auc(roc_boost_mushroomShape)

auc_results <- data.frame(
  Model = c("tree.mushroom", "tree.mushroomColor", "tree.mushroomShape",
            "boost.mushroom", "boost.mushroomColor", "boost.mushroomShape"),
  AUC = c(auc_tree_mushroom, auc_tree_mushroomColor, auc_tree_mushroomShape,
          auc_boost_mushroom, auc_boost_mushroomColor, auc_boost_mushroomShape)
)

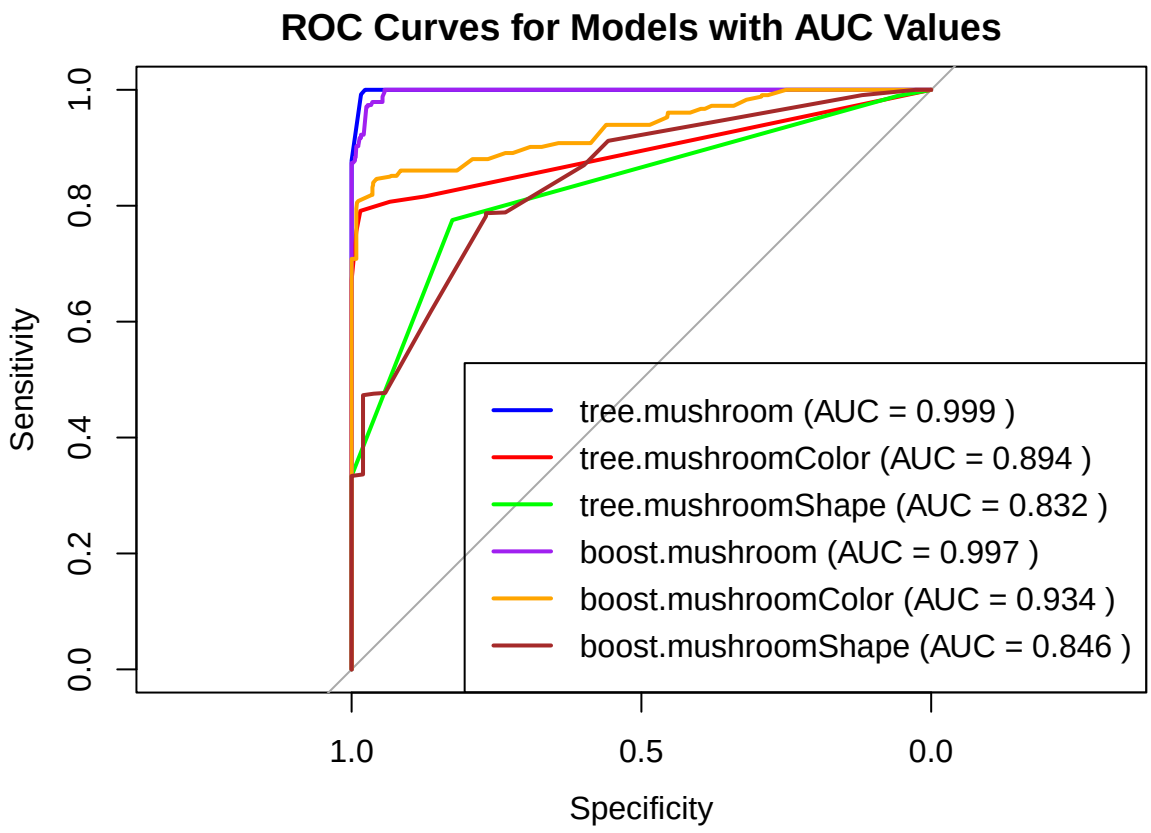
print(auc_results)
```

```
##                 Model       AUC
## 1       tree.mushroom 0.9989174
## 2  tree.mushroomColor 0.8941214
## 3  tree.mushroomShape 0.8319152
## 4      boost.mushroom 0.9970064
## 5 boost.mushroomColor 0.9339232
## 6 boost.mushroomShape 0.8460372
```

```r
plot(roc_tree_mushroom, col = "blue", lwd = 2, main = "ROC Curves for Models with AUC Values")

# Add the other ROC curves
lines(roc_tree_mushroomColor, col = "red", lwd = 2)
lines(roc_tree_mushroomShape, col = "green", lwd = 2)
lines(roc_boost_mushroom, col = "purple", lwd = 2)
lines(roc_boost_mushroomColor, col = "orange", lwd = 2)
lines(roc_boost_mushroomShape, col = "brown", lwd = 2)

# Add a legend with AUC values
legend("bottomright",
       legend = c(paste("tree.mushroom (AUC =", round(auc_tree_mushroom, 3), ")"),
                  paste("tree.mushroomColor (AUC =", round(auc_tree_mushroomColor, 3), ")"),
                  paste("tree.mushroomShape (AUC =", round(auc_tree_mushroomShape, 3), ")"),
                  paste("boost.mushroom (AUC =", round(auc_boost_mushroom, 3), ")"),
                  paste("boost.mushroomColor (AUC =", round(auc_boost_mushroomColor, 3), ")"),
                  paste("boost.mushroomShape (AUC =", round(auc_boost_mushroomShape, 3), ")")),
       col = c("blue", "red", "green", "purple", "orange", "brown"),
       lwd = 2)
```



Model analysis for RF Model

```
train_predictions <- predict(rf_model, train)
train_conf_matrix <- confusionMatrix(train_predictions, train$poisonous)
print(train_conf_matrix)
```

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction  Edible Poisonous
##   Edible      3345         0
##   Poisonous      0      3155
##
##                Accuracy : 1
##                  95% CI : (0.9994, 1)
##     No Information Rate : 0.5146
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##             Sensitivity : 1.0000
```

```
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 1.0000
##              Prevalence : 0.5146
##          Detection Rate : 0.5146
##    Detection Prevalence : 0.5146
##       Balanced Accuracy : 1.0000
##
##        'Positive' Class : Edible
##
```

```
test_predictions <- predict(rf_model, test)
test_conf_matrix <- confusionMatrix(test_predictions, test$poisonous)
print(test_conf_matrix)
```

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction  Edible Poisonous
##   Edible       863         0
##   Poisonous      0       761
##
##                Accuracy : 1
##                  95% CI : (0.9977, 1)
##     No Information Rate : 0.5314
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##             Sensitivity : 1.0000
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 1.0000
##              Prevalence : 0.5314
##          Detection Rate : 0.5314
##    Detection Prevalence : 0.5314
##       Balanced Accuracy : 1.0000
##
##        'Positive' Class : Edible
##
```

We see that the training accuracy equals one. This indicates to me that there is overfitting in this model. The test accuracy is also one which indicates to me that the data set is easy to classify.

```
set.seed(123)

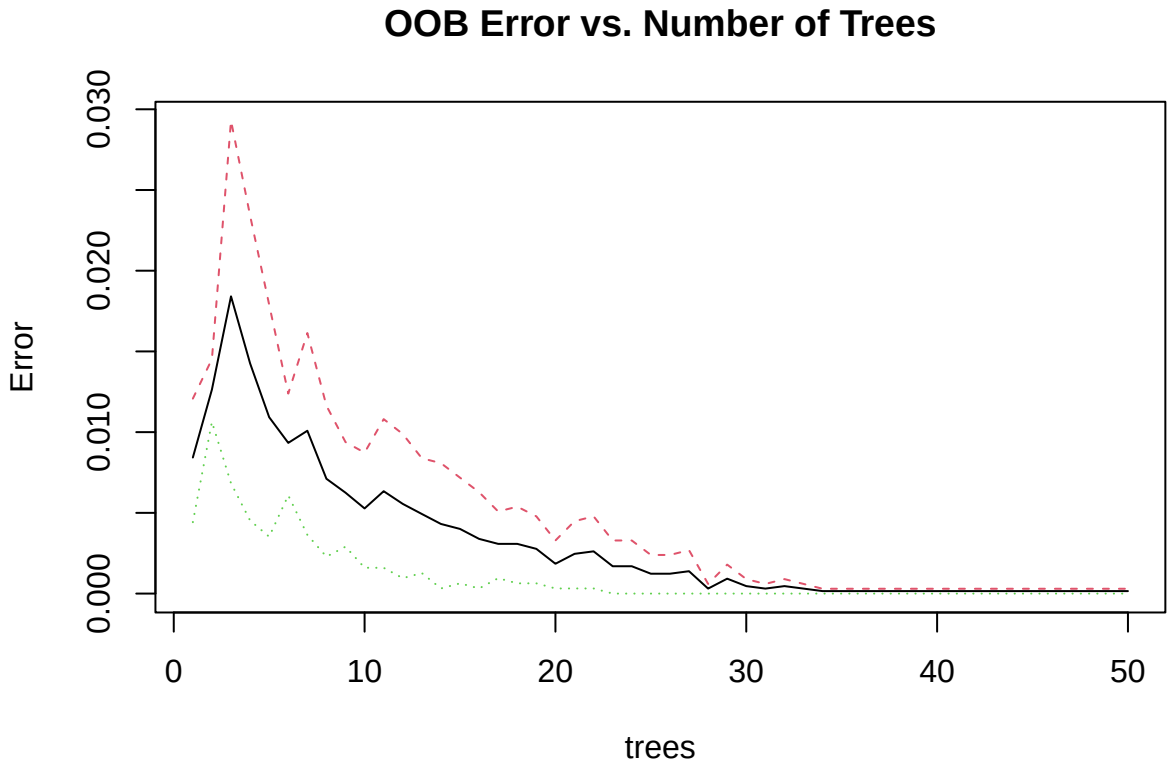
rf_model <- randomForest(poisonous ~ .,
                         data = train,
                         ntree = 50,
                         mtry = 3,
                         importance = TRUE,
```

```
                         keep.inbag = TRUE)

# Print model summary
print(rf_model)
```

```
##
## Call:
##  randomForest(formula = poisonous ~ ., data = train, ntree = 50,      mtry = 3, importance = TRUE, ke
##                Type of random forest: classification
##                      Number of trees: 50
## No. of variables tried at each split: 3
##
##         OOB estimate of  error rate: 0.02%
## Confusion matrix:
##           Edible Poisonous  class.error
## Edible      3344         1 0.0002989537
## Poisonous      0      3155 0.0000000000
```

```
# Extract and plot OOB error
plot(rf_model, main = "OOB Error vs. Number of Trees")
```



We can see as more trees are added past around 30-40 the error flattens to zero so adding more trees will not improve model accuracy.

Cross Validation for RF model

```r
set.seed(123)

cv_control <- trainControl(method = "cv", number = 10)

rf_cv_model <- train(
  poisonous ~ .,
  data = mushrooms,
  method = "rf",
  trControl = cv_control,
)

print(rf_cv_model)
```

```
## Random Forest
##
## 8124 samples
##   19 predictor
##    2 classes: 'Edible', 'Poisonous'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 7311, 7311, 7311, 7311, 7312, 7313, ...
## Resampling results across tuning parameters:
##
##   mtry  Accuracy   Kappa
##    2    0.9137185  0.8262191
##   42    1.0000000  1.0000000
##   83    1.0000000  1.0000000
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 42.
```

```r
cv_results <- rf_cv_model$resample
print(cv_results)
```

```
##    Accuracy Kappa Resample
## 1         1     1   Fold02
## 2         1     1   Fold01
## 3         1     1   Fold04
## 4         1     1   Fold03
## 5         1     1   Fold06
## 6         1     1   Fold05
## 7         1     1   Fold08
## 8         1     1   Fold07
## 9         1     1   Fold10
## 10        1     1   Fold09
```

```r
mean_acc <- mean(cv_results$Accuracy)
sd_acc <- sd(cv_results$Accuracy)
```

```r
cat("Mean Accuracy:", mean_acc, "\n")
```

```
## Mean Accuracy: 1
```

```r
cat("Standard Deviation of Accuracy:", sd_acc, "\n")
```

```
## Standard Deviation of Accuracy: 0
```

Results

```r
mushroomTPR <- mushroomCM[1]/(mushroomCM[1]+mushroomCM[2])
mushroomFPR <- mushroomCM[3]/(mushroomCM[3]+mushroomCM[4])
colorTPR <- colorCM[1]/(colorCM[1]+colorCM[2])
colorFPR <- colorCM[3]/(colorCM[3]+colorCM[4])
shapeTPR <- shapeCM[1]/(shapeCM[1]+shapeCM[2])
shapeFPR <- shapeCM[3]/(shapeCM[3]+shapeCM[4])
boostTPR <- boostCM[1]/(boostCM[1]+boostCM[2])
boostFPR <- boostCM[3]/(boostCM[3]+boostCM[4])
boostColorTPR <- boostColorCM[1]/(boostColorCM[1]+boostColorCM[2])
boostColorFPR <- boostColorCM[3]/(boostColorCM[3]+boostColorCM[4])
boostShapeTPR <- boostShapeCM[1]/(boostShapeCM[1]+boostShapeCM[2])
boostShapeFPR <- boostShapeCM[3]/(boostShapeCM[3]+boostShapeCM[4])

errordf <- data.frame(TPR=c(mushroomTPR,colorTPR,shapeTPR,boostTPR,boostColorTPR,boostShapeTPR),FPR=c(mu
library(knitr)
kable(errordf)
```

	TPR	FPR
Mushroom	0.9929825	0.0182055
Color	0.8424182	0.0211382
Shape	0.8065611	0.2027027
Mushroom(Boost)	0.9811321	0.0399485
Color(Boost)	0.8538539	0.0160000
Shape(Boost)	0.7997587	0.2515723

Conclusion

From our model testing and results, we conclude that the decision tree is the best model for classifying mushroom toxicity, with a False Positive Rate of 0.0182055. We found that the random forest model overfits the data so we looked at the next best model, which was the decision tree. The boosted decision tree, came close to the normal decision tree but fell short in the AUC/ROC test and FPR values. All in all, we found that a normal decision tree is better than both boosting and a random forest for this scenario. The random forest was overfitting the data and the normal decision tree had slightly better test results than the boosted tree. We also saw that pruning the decision tree did not have an impact at all, signifying that there isn't an overfitting issue.

Now back to the initial question. We found that while gill color is an important variable when classifying a mushroom's toxicity, it isn't as impactful as spore print color. Additionally, when we think of a mushroom's color, we think of the cap color, not the color of the gills. Thus, to answer the questions proposed in the introduction, the best indicators of toxicity are spore print color, gill color, and ring type. Contrary to popular belief, the color of the cap is not too important when it comes to determining if a mushroom is poisonous or not. Even in the model that utilized only color type variables, color cap came in fourth out of the five variables used in prediction. Next time when you see an unidentified mushroom, perhaps check the color of the spore prints, and not make a judgement based on solely the outer appearance of the mushroom.